

Actividad N°2 — Cifrado, hashing y tokenización de datos sensibles

Nombre: Grupo MTG Companion
Integrantes: Vicente Fuentes, Tomás Rodríguez, Tomás Solano
Fecha: 2 oct 2026
Proyecto: MTG Companion — app de intercambio/compraventa de cartas (apps/), backend NestJS + Prisma + PostgreSQL
Repositorio: https://github.com/inzenfenix/MTGCompanion_TCG · tarea: [issue #35](#)
Insumo: diccionario de datos de la Actividad N°1 → [MAPEO_CIA.md](#); revisión campo por campo → [CIFRADO_CAMPOS.md](#)

La app es, en esencia, un marketplace de cartas: lo que vale la pena proteger son los **datos del jugador** (credenciales, segundo factor, ubicación, pagos y sesión). El resto (catálogo de cartas, publicaciones, cupones) es información pública o cuyo riesgo es de integridad, no de confidencialidad.

Pregunta guía para elegir la técnica: **¿el sistema necesita recuperar el dato original?** Solo comparar → *hashing* · recuperarlo → *cifrado* · no tocarlo nunca → *tokenización* · que no lo falsifiquen → *firma*.

ITEM I — Cuadro por dato sensible

CAMPO	NECESIDAD	ELECCIÓN	CRITERIO TÉCNICO	IMPLEMENTACIÓN	EVIDENCIA
Contraseña (<code>User.passwordHash</code>)	Solo comparar en el login; nunca recuperarla.	Hashing (bcrypt)	Dato de baja entropía → hash lento y con sal contra fuerza bruta offline si se filtra la BD. Costo 12, sal aleatoria por usuario embebida en el hash.	<code>bcryptjs</code> en <code>UserService.register()</code> / <code>validateCredentials()</code> (users.service.ts)	04 paso 2: la BD guarda <code>\$2b\$12\$...</code>
Token de renovación de sesión (<code>RefreshToken.tokenHash</code>)	Solo verificar que existe y no fue usado.	Hashing (SHA-256)	Aleatorio de 512 bits y de un solo uso → alta entropía, basta un hash rápido sin sal. Se rota en cada uso.	<code>node:crypto</code> en <code>AuthService.hashToken()</code> (auth.service.ts)	02 : <code>auth-refresh.e2e-spec</code> (rotación, reuso rechazado)

CAMPO	NECESIDAD	ELECCIÓN	CRITERIO TÉCNICO	IMPLEMENTACIÓN	EVIDENCIA
Segundo factor (<code>UserSettings.twoFactorSecret</code> , secreto TOTP)	Recuperar el secreto original para calcular el código de 6 dígitos en cada login.	Cifrado simétrico (AES-256-GCM)	Un hash no sirve (TOTP necesita el secreto). GCM = cifrado autenticado : si alguien altera el valor, el descifrado falla. IV aleatorio de 12 bytes por escritura; prefijo de versión de llave para rotarla.	<code>FieldEncryptionService</code> usado en <code>PrismaUserRepository</code> (cifra al escribir, descifra al leer)	01 · 03 · 04 pasos 3-6 y 10
Ubicación del jugador (<code>UserSettings.lastLat</code> / <code>lastLng</code>)	Recuperar para calcular la distancia al vendedor en la búsqueda. Dato personal (Ley 21.719).	Cifrado simétrico (AES-256-GCM)	La distancia se calcula en el backend (Node), así que se puede descifrar ahí; las coordenadas nunca vuelven al cliente, solo <code>distanceKm</code> . Misma llave que el 2FA.	Migración <code>Float</code> → <code>TEXT</code> (<code>migration.sql</code>); cifrado en <code>PrismaUserRepository</code> , descifrado en <code>PrismaCardRepository</code>	03 antes/después · 04 pasos 7 y 9
Datos de pago (tarjeta)	No tocarlos nunca ; solo referenciar el pago.	Tokenización	<code>MercadoPago</code> es la bóveda: la tarjeta se ingresa en su checkout y el sistema solo guarda <code>paymentRef</code> (el token) para consultar el estado.	SDK oficial <code>mercadopago</code> (<code>mercadopago-payment.provider.ts</code>); <code>Transaction</code> solo tiene <code>paymentRef</code> en <code>schema.prisma</code>	<code>schema.prisma</code> : ninguna columna de tarjeta; 02 flujos de pago
Sesión (access token JWT)	Que nadie pueda fabricar un token válido (integridad).	Firma (HMAC-SHA256)	Lo importante no es ocultarlo sino verificar quién lo emitió. El secreto de firma ya no tiene valor por defecto en producción.	<code>@nestjs/jwt</code> HS256; <code>JWT_SECRET</code> desde AWS Secrets Manager; <code>assertProductionSecrets()</code>	04 paso 6 · 05 paso 5
Código de verificación de la compraventa (QR)	Que el QR apunte a una publicación real y no se pueda falsificar.	Firma (HMAC-SHA256)	Token firmado con <code>typ: 'listing'</code> y vencimiento de 3 minutos; el backend valida la firma al escanearlo.	<code>listing-token.ts</code>	02: <code>listing-token.e2e-spec</code>

CAMPO	NECESIDAD	ELECCIÓN	CRITERIO TÉCNICO	IMPLEMENTACIÓN	EVIDENCIA
Email y nombre visible (User.email, displayName)	El email se busca por igualdad en el login; el nombre se muestra a la contraparte.	Sin cifrado de campo	Cifrar con IV aleatorio impediría buscar por email (@unique); el nombre es público por diseño. Se protegen en tránsito (TLS) y en reposo a nivel de disco.	—	Justificación en CIFRADO_CAMPOS.md §2

Fuera del alcance de esta entrega (infraestructura, ya diseñado en [CIFRADO_CAMPOS.md](#) y registrado en [ROADMAP.md](#), workstream O): TLS delante del backend (O5/I6), cifrado del disco de Postgres y SSE explícito en S3 (O6), y endurecimientos menores (O8).

ITEM II — Implementación

1. Código fuente

Archivo	Qué hace
common/crypto/field-encryption.service.ts	AES-256-GCM con node:crypto (sin criptografía propia). Formato v1:<iv>:<tag>:< ciphertext>, IV aleatorio por escritura, anillo de llaves por versión.
common/crypto/crypto.module.ts	Módulo global; se niega a arrancar sin <code>FIELD_ENCRYPTION_KEY</code> (no hay llave por defecto ni en desarrollo).
users/infrastructure/prisma-user.repository.ts	Cifra <code>twoFactorSecret</code> , <code>lastLat</code> , <code>lastLng</code> al escribir y los descifra al leer. Es la única capa que ve el texto cifrado: dominio y controladores trabajan con texto plano.
cards/infrastructure/prisma-card.repository.ts	Descifra la ubicación del vendedor solo para calcular la distancia.
prisma/migrations/20261002120000_encrypt_location_fields	<code>lastLat/lastLng</code> pasan de <code>DOUBLE PRECISION</code> a <code>TEXT</code> .
scripts/encrypt-existing-fields.ts	Cifra las filas antiguas en texto plano y re-cifra las de una llave retirada (rotación). Idempotente; corre en cada arranque del contenedor (docker-entrypoint.sh).
config/configuration.ts	Lee las llaves del entorno y bloquea el arranque en producción si falta <code>JWT_SECRET</code> o <code>FIELD_ENCRYPTION_KEY</code> .
common/crypto/field-encryption.service.spec.ts	Pruebas unitarias (ida y vuelta, alteración, llave incorrecta, IV aleatorio, rotación, arranque sin secretos).
infra/terraform/secrets.tf , variables.tf , scripts/deploy-backend.sh	La llave vive en AWS Secrets Manager y llega al contenedor del backend al desplegar.

2. Protección de los campos críticos

- **Cifrados en reposo (AES-256-GCM):** secreto 2FA y ubicación del jugador. En la BD solo queda `v1:5d1x...:Qm9o...:4FvA...`; un volcado de Postgres no revela ni el secreto ni las coordenadas.
- **Hasheados:** contraseña (bcrypt, costo 12) y token de renovación (SHA-256).
- **Tokenizados:** datos de tarjeta, que nunca entran al sistema (MercadoPago).
- **Firmados:** token de sesión y QR de compraventa (HMAC-SHA256).

Gestión de llaves

Recomendación	Cómo se cumple
Nunca junto al dato	La llave está en la EC2 del backend (léida desde Secrets Manager); los datos en la EC2 de Postgres. La BD nunca ve la llave.
Nunca en el código	Solo variables de entorno. Sin valor por defecto: sin llave, el backend no arranca.
Separación	Llave de cifrado $\neq$ secreto JWT $\neq$ secretos de MercadoPago.
Rotación	Prefijo de versión en cada valor; la llave anterior se deja en <code>FIELD_ENCRYPTION_RETIRED_KEYS</code> y <code>npm run db:encrypt-fields</code> re-cifra todo con la nueva.
Pérdida	Perder la llave hace ilegibles los campos cifrados $\rightarrow$ se guarda solo en Secrets Manager (y una copia del administrador fuera del repo).

3. Evidencia de funcionamiento

#	Evidencia	Qué demuestra
01	<code>01_tests_unitarios.txt</code>	13/13 pruebas: ida y vuelta, IV distinto en cada cifrado, alteración del texto o del tag rechazada, llave incorrecta rechazada, rotación, arranque sin secretos.
02	<code>02_tests_e2e.txt</code>	38/38 pruebas e2e del backend pasan con los campos ya cifrados: nada se rompió.
03	<code>03_bd_antes_despues.txt</code>	La ubicación pasó de número en texto plano a <code>v1:...</code> ; el script de migración es idempotente; 0 valores en texto plano.
04	<code>04_flujo_cifrado.txt</code>	Flujo real con <code>curl</code> : alta de 2FA, login con 2FA (código malo $\rightarrow$ 401, bueno $\rightarrow$ sesión), ubicación cifrada y búsqueda con distancia, y valor alterado en la BD $\rightarrow$ el backend lo rechaza.
05	<code>05_llaves_no_expuestas.txt</code>	Ver punto 5.

Para reproducirla: `apps/backend/docker/dev.sh up`, `npm run build && node dist/main.js`, y luego `flujo_cifrado.sh` y `llaves_no_expuestas.sh`.

4. Cómo se recupera o valida cada dato

Dato	Técnica	Cómo se usa después
Contraseña	Hash bcrypt	<code>bcrypt.compare(ingresada, hash)</code> — se valida , nunca se recupera.
Token de renovación	Hash SHA-256	Se hashea el token presentado y se busca por <code>tokenHash</code> ; se borra y se emite uno nuevo.
Secreto 2FA	AES-256-GCM	El repositorio lo descifra con la llave en memoria justo antes de validar el código TOTP. Si el valor fue alterado, el tag GCM no calza y el login falla (cerrado).
Ubicación	AES-256-GCM	El repositorio la descifra solo para calcular <code>distanceKm</code> ; al cliente nunca le llegan coordenadas.
Pago	Tokenización	Se consulta a MercadoPago por <code>paymentRef</code> ; la tarjeta nunca vuelve al sistema.
Sesión / QR	Firma HMAC	Se valida la firma con el secreto; si no calza → 401.

5. Las llaves y secretos no quedan expuestos

Detalle en [05_llaves_no_expuestas.txt](#):

1. **gitleaks** sobre las 180 commits del historial: *no leaks found*. Los 9 hallazgos iniciales fueron revisados uno por uno: contraseñas de cuentas desechables que crean y borran los tests e2e; quedan justificados en `.gitleaksignore`.
2. `.env`, `terraform.tfvars` y `*.tfstate` están ignorados por git y **ningún commit** los incluyó; solo se versionan los `*.example` sin valores.
3. La llave de cifrado local aparece **0 veces** en `git log --all -p`.
4. Postgres guarda solo texto cifrado; la llave nunca llega a la BD.
5. El backend **se niega a arrancar** en producción sin `JWT_SECRET` o `FIELD_ENCRYPTION_KEY`, y tampoco arranca en desarrollo sin llave de cifrado. Los mensajes de error nombran la variable, nunca su valor.